

MSc Data Science

Big Data Analytics Technologies Coursework

Chamika Deshan

COMScDS251P-015

Setup

The source code for all tasks can be found in git repository below.

https://github.com/deshancha/bigdata_cw

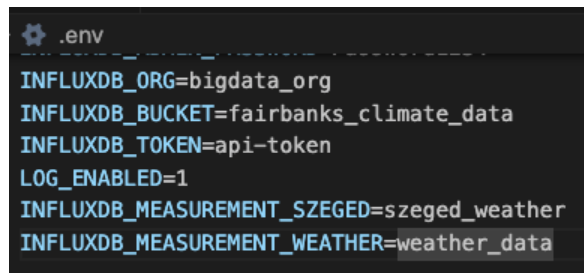
Adjusting the parameter below in .env file can Disable/Enable Logs, Default is enabled.

LOG_ENABLED=1

Task 1: Distributed Time-Series Data Management using InfluxDB

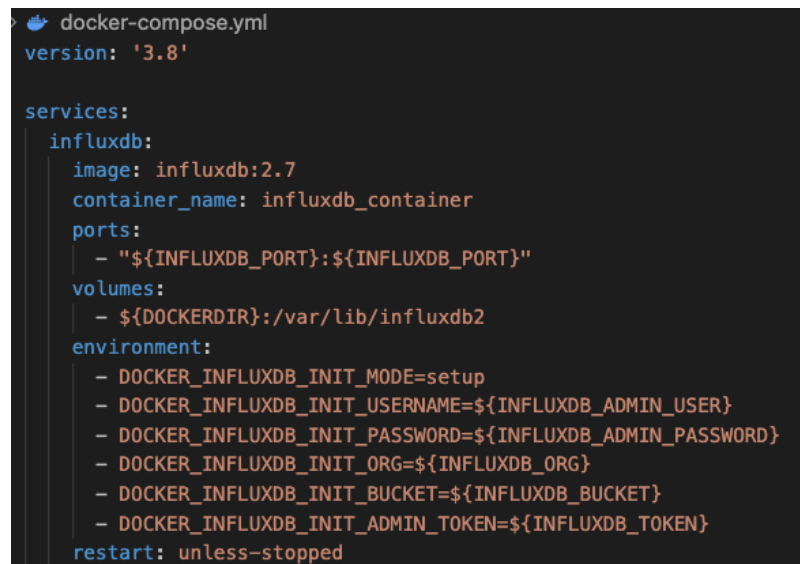
Step 1.1 Containerized Environment Provisioning

Figure 1.1 below illustrates .env configurations and figure 1.2 below depicts the docker compose file



```
.env
INFLUXDB_ORG=bigdata_org
INFLUXDB_BUCKET=fairbanks_climate_data
INFLUXDB_TOKEN=api-token
LOG_ENABLED=1
INFLUXDB_MEASUREMENT_SZEGED=szeged_weather
INFLUXDB_MEASUREMENT_WEATHER=weather_data
```

Figure 1.1: .env file



```
docker-compose.yml
version: '3.8'

services:
  influxdb:
    image: influxdb:2.7
    container_name: influxdb_container
    ports:
      - "${INFLUXDB_PORT}:${INFLUXDB_PORT}"
    volumes:
      - ${DOCKERDIR}:/var/lib/influxdb2
    environment:
      - DOCKER_INFLUXDB_INIT_MODE=setup
      - DOCKER_INFLUXDB_INIT_USERNAME=${INFLUXDB_ADMIN_USER}
      - DOCKER_INFLUXDB_INIT_PASSWORD=${INFLUXDB_ADMIN_PASSWORD}
      - DOCKER_INFLUXDB_INIT_ORG=${INFLUXDB_ORG}
      - DOCKER_INFLUXDB_INIT_BUCKET=${INFLUXDB_BUCKET}
      - DOCKER_INFLUXDB_INIT_ADMIN_TOKEN=${INFLUXDB_TOKEN}
    restart: unless-stopped
```

Figure 1.2: Docker Compose File

Step 1.2: Database Schema & Ingestion Scripting

This is a Python program written with clean architecture with dependency injection. Below are the usecases

1. Load Data
2. Crate Bucket
3. Drop Bucket
4. Open Connection
5. Close Connection
6. Load CSV
7. Write Records – Writing to Database
8. Writing Records

We write both `szeged_weather` and `weather_data` to same bucket. We destinguished them by measurement as depicted in the code segment in figure 1.3 below.

```
self.measurement_szeged = os.environ.get("INFLUXDB_MEASUREMENT_SZEGED", "szeged_weather")
self.measurement_weather = os.environ.get("INFLUXDB_MEASUREMENT_WEATHER", "weather_data")
```

Figure 1.3: Measurements to distiguish Points

Program Execution

```
python3 src/main.py <PARAM>
```

PARAM	Description
-1	Drop Bucket configured in .env INFLUXDB_BUCKET
0	Crete Bucket
1	Write weatherHistory.csv as 1000 batches
2	Write Project 1 - Weather Dataset.csv as 1000 batches

Table 1.1 : Program execution

Maintain the Historical TimeStamp

```
# 2006-04-01 00:00:00.000 +0200
date_str = rec.get("Formatted Date")
dt = datetime.strptime(date_str, "%Y-%m-%d %H:%M:%S.%f %Z")
point.time(dt, WritePrecision.S)
```

Figure 1.4: Keep Historial Timestamp

Step 1.3: Flux Time-Series Analytical Querying

Window Aggregation to compute sliding hourly averages across an extended observation span.

Here we group all data points into 1 hour windows and computes the mean temperature for each window. And we get a time series of hourly temperatures for Szeged from 2016 January to December. Figure 1.5 below depicts the related Influx query.

```
from(bucket: "fairbanks_climate_data")
// Szeged Dat Range
|> range(start: 2006-01-01T00:00:00Z, stop: 2016-12-31T23:59:59Z)
|> filter(fn: (r) => r["_measurement"] == "szeged_weather")
|> filter(fn: (r) => r["_field"] == "temperature")
// Groups to 1 hr window and get average
|> aggregateWindow(every: 1h, fn: mean)
```

Figure 1.5: Flux time-series query

Anomaly Isolation filters to capture observations exceeding two standard deviations from the dataset mean. Figure 1.6 below depicts the related Influx Query.

```
data = from(bucket: "fairbanks_climate_data")
|> range(start: 2012-01-01T00:00:00Z, stop: 2012-12-31T23:59:59Z)
|> filter(fn: (r) => r["_measurement"] == "weather_data")
|> filter(fn: (r) => r["_field"] == "temperature")
// find mean and std
mean = (data |> mean() |> findRecord(fn: (key) => true, idx: 0))._value
std = (data |> stddev() |> findRecord(fn: (key) => true, idx: 0))._value
meanPlusTwo = mean + 2.0 * std
meanMinusTwo = mean - 2.0 * std
// Filter mean +-2 anomalies
data
|> filter(fn: (r) => r._value > (meanPlusTwo) or r._value < (meanMinusTwo))
```

Figure 1.6: Influx query for anomaly isolation

Finally We put visualize in the dashbaord as shown in figure 1.7 below

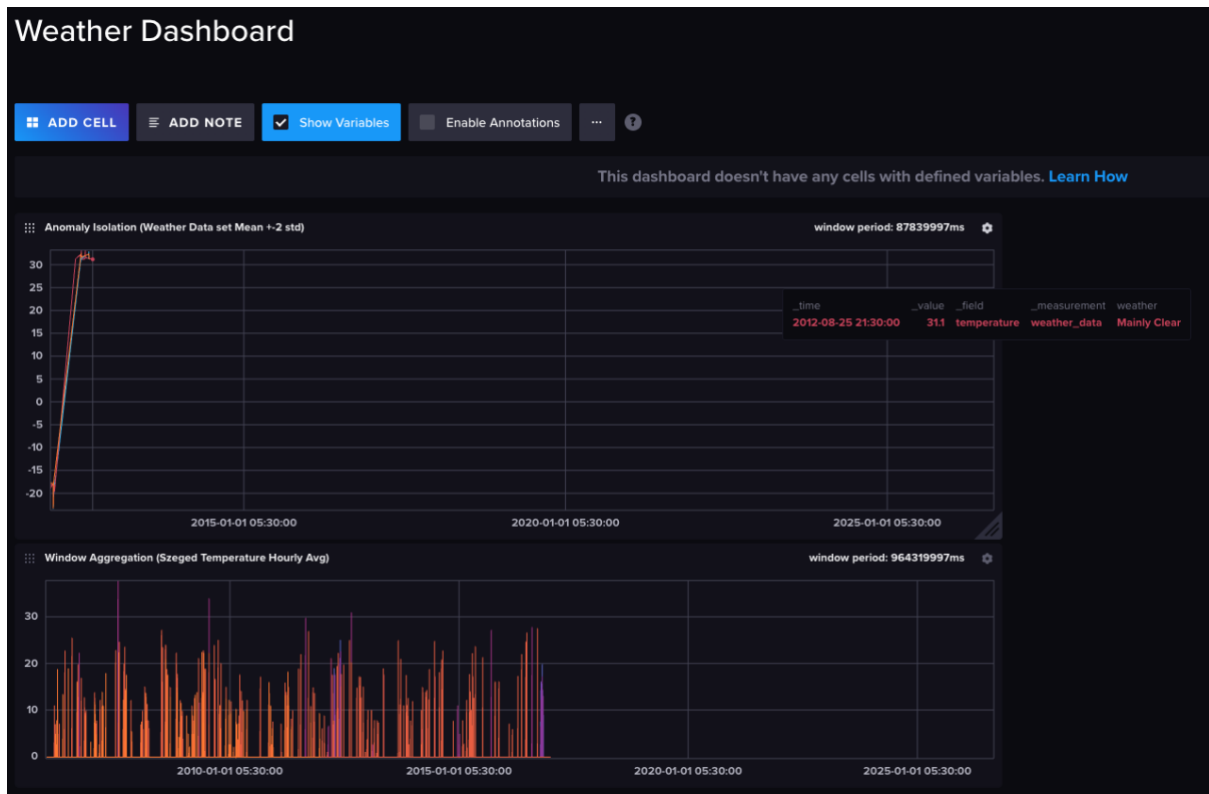


Figure 1.7: Dashboard Visualization

Downsampling tasks that continuously summarize historical data into an auxiliary bucket governed by an explicit 30-day retention rule

Figure 1.8 depicts the downsampling task runs everyday 1 hour after midnight and get daily average data and write to separate bucket named "weather_data_downsampled", the bucket has 30 day retention policy as shown in figure 1.9 (>30 days automatically get deleted).

```
option task = {name: "Daily Weather Downsampling Task", every: 1d, offset: 1h}

from(bucket: "fairbanks_climate_data")
  // Get data from last day
  |> range(start: -1d)
  |> filter(fn: (r) => r["_measurement"] == "szeged_weather")
  // group hourly data to daily avg
  |> aggregateWindow(every: 1d, fn: mean)
  // Write to 30 day auxiliary bucket
  |> to(bucket: "weather_data_downsampled")
```

Figure 1.8: Scheduled Task

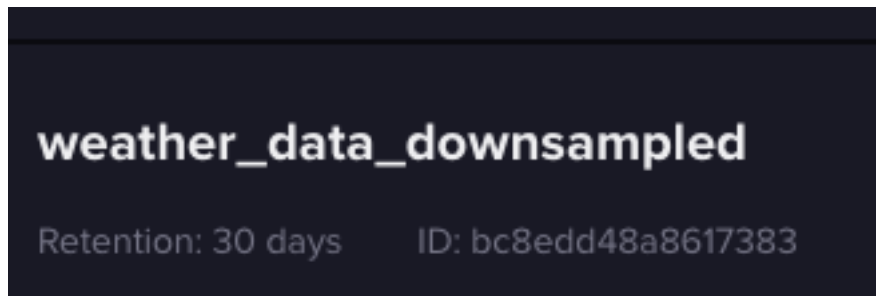


Figure 1.9: Retention period 30 days

Task 2: Real-Time Stream Ingestion & Processing with Apache Kafka and Apache Flink

Components

1. Live Telemetry Data Producer
2. Flink Processing and Sink to InfluxDB
3. Influx Time Series Traffic Data Visualize

Implmentation with Python and Kotlin on containerlized environment with clean architecture with proper dependency injection for scalability.

Step 2.1: Multi-Container Orchestration Setup

Figure 2.1 below illustrates .env configurations and figure 2.2 below is the docker container.

```
# Flink
FLINK_PORT=8081
# 1 slot(worker thread) -> if many logs could be messy for explanation
FLINK_TASK_SLOTS=1
FLINK_VERSION=1.18.1

# Kafka
KAFKA_PORT=29092
KAFKA_UI_PORT=8089
KAFKA_TOPIC=traffic-telemetry

# Producer
TELEMETRY_API_URL=https://data.austintexas.gov/resource/sh59-i6y9.json
TELEMETRY_API_LIMIT=1000
LOG_ENABLED=1

# Influx Config
DOCKERDIR=./influxdb_data
INFLUXDB_HOST=localhost
INFLUXDB_PORT=8087
INFLUXDB_INTERNAL_PORT=8086
INFLUXDB_ADMIN_USER=admin
INFLUXDB_ADMIN_PASSWORD=Password1234
INFLUXDB_ORG=bigdata_cw
INFLUXDB_BUCKET=traffic_data
INFLUXDB_TOKEN=api-token
```

Figure 2.1: .env config for docker and programs

☐	☒	task2	-	-	-	5.05%	28 r				
☐	☒	influxdb	8f7747a3628d	influxdb:2.	8087:8086	0.03%	28 r				
☐	☒	jobmanager	9b8e40f759f5	task2:jobm	8081:8081	1.34%	28 r				
☐	☒	kafka-ui	e027e9a53e9d	kafbat/kafl	8089:8080	0.15%	28 r				
☐	☒	kafka	d3802dc22540	confluentir	29092:29092	1.97%	28 r				

Figure 2.2: Docker Container

Step 2.2: Kafka Topic Architecture & Live Simulation

This is python implementation for retrieve traffic telemetry data from camera traffic api and send to kafka topic “traffic-telemetry” every 2 second.

Usecases

1. Fetch Telemetry – Fetch Records
2. Parse Telemetry – Api Data to Json
3. Connect Producer – Connect to Kafka broker
4. Send Telemetry – Send Json to Kafka topic
5. Close Producer – Closes and Release resources

Above are separately distributed to respective usecases with usecase names as depicted in figure 2.3 below

```
class ParseTelemetryUseCase:
    def __init__(self, logger):
        self.logger = logger

    def execute(self, row: Dict[str, Any]) -> Dict[str, Any]:
        return {
            "camera_id": row.get("atd_device_id", "unknown_sensor"),
            "timestamp": row.get("read_date", datetime.utcnow().isoformat()) +
            "direction": row.get("direction", "Unknown"),
            "vehicle_count": int(row.get("volume", 0))
        }
```

Figure 2.3: Parsing Telemetry api data

Step 2.3: Stateful Flink Processing & Out-of-Order Handling

This is a kotlin Flink stream processing implementation that source is consume from Kafka Topic “traffic-telemetry” and aggregate traffic data in 10 minutes window frames and sink the result to InfluxDB.

Watermark Strategy and Timing

We measure the progress of time at stream level not at the source level, therefore the kafka source is initialized with No Watermark Strategy.

After we receive the Json payload we assign timestamp and watermark strategy allowing 10 seconds delay (leaving space for network delays). Finally stream key assigned with camera id and they grouped into 10 minutes window for summing up the vehicle count as shown in figure 2.4 below.


```

val watermarkStrategy = WatermarkStrategy
    // 10 sec delay allow
    .forBoundedOutOfOrderness<TelemetryRecord>(Duration.ofSeconds(10))
    // watermark strategy telemetryRecord->timeStamp we set above, we set noWatermarks
    .withTimestampAssigner { telemetryRecord, _ -> telemetryRecord.timeStamp }

val watermarkedStream = parsedStream.assignTimestampsAndWatermarks(watermarkStrategy)

val summingUp = watermarkedStream
    .keyBy { it.camId }
    // 10 min window, seconds for easy monitoring
    .window(TumblingEventTimeWindows.of(Time.minutes(10)))

```

Figure 2.4: Watermark strategy and Windowing

Deploying to Flink

We have created a kotlin helper Task to build jar and deploy to Job Manager as shown in figure 2.5 below. We can just execute “gradle deployToDocker” and jars would be build, copy to job manager and execute.

```

tasks.register<Exec>("deployToDocker") {
    dependsOn("shadowJar")
    group = "deployment"
    description = "Copy jar to Flink JobManager"

    commandLine("docker", "cp", "${tasks.shadowJar.get().archiveFile.get().asFile.absolutePath}", "task2-jobmana

    doLast {
        // Flink job submit
        val jarName = tasks.shadowJar.get().archiveFile.get().asFile.name
        ProcessBuilder(
            "docker", "exec", "task2-jobmanager",
            "flink", "run", "-d", "-c", "com.bigdata.cw.processor.TelemetryProcessorApp",
            "/opt/flink/${jarName}"
        )
        .inheritIO()
        .start()
        .waitFor()
    }
}

```

Figure 2.5: Gradle Task for execute Flink Processing with Job Manager

For the sink of the stream we added a InfluxDB sink and write the data to InfluxDB as shown in figure 2.6 below.

```

class FlinkInfluxDbSink(private val logger: ILogger) : RichSinkFunction<String>() {
    private var client: InfluxDBClient? = null

    override fun invoke(value: String, context: SinkFunction.Context) {
        try {
            val json = Gson().fromJson(value, JsonObject::class.java)
            val camId = json.get("cam_id")?.asString ?: "unknown"
            val count = json.get("vehicle_count")?.asInt ?: 0
            val windowStart = json.get("original_start")?.asLong ?: System.currentTimeMillis()

            %L for Agent
            val point = Point.measurement("camera_traffic")
                .addTag("cam_id", camId)
                .addField("vehicle_count", count)
                .time(Instant.ofEpochMilli(windowStart), WritePrecision.MS)

            client?.writeApiBlocking?.writePoint(point)
        }
    }
}

```

Figure 2.6: InfluxDBSink

Finally we could visualize the traffic data time series using a Influx query as shown in Traffic Dashboard figure 2.7 below.

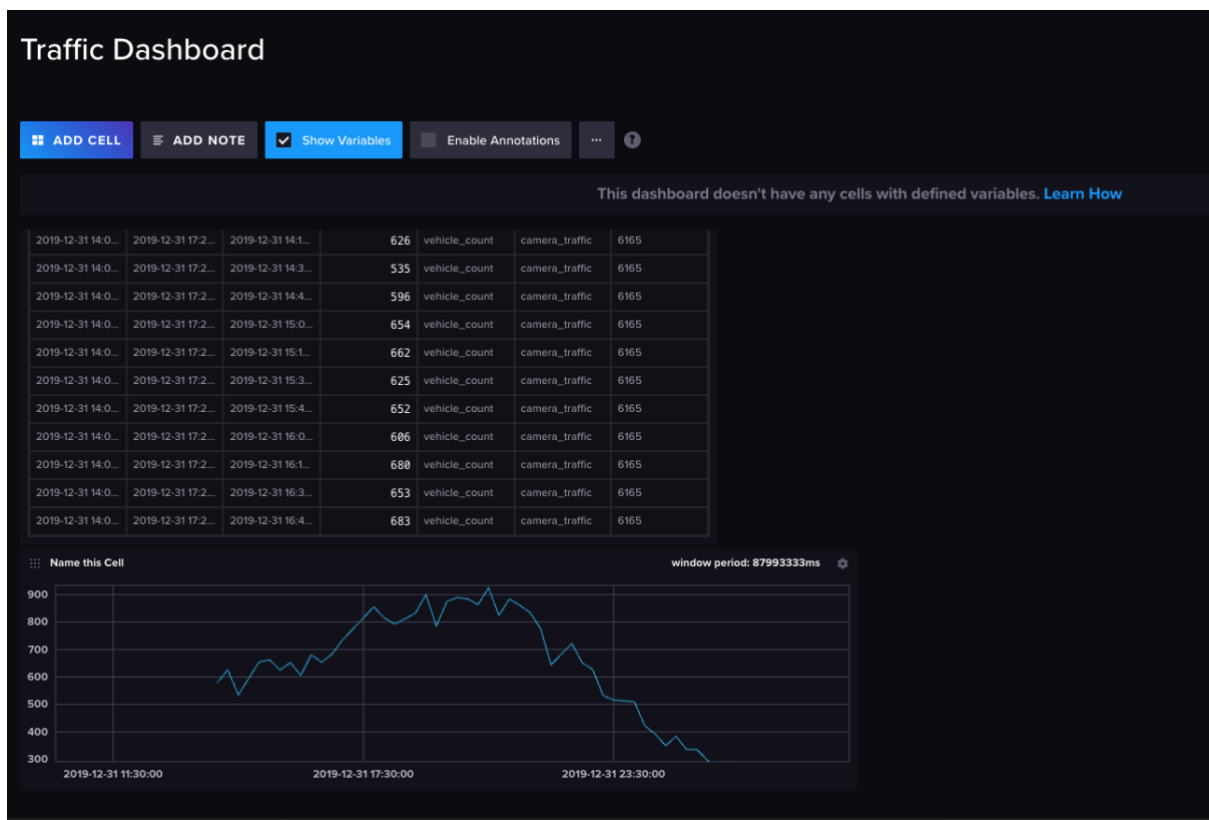


Figure 2.7: Traffic Dashboard

Task 3: Scalable Data Analytics with Apache Spark

Here we have a dataset which maps the hyperlinks pointing between web pages of Berkley and Stanford university domains.

Step 3.1: Spark Container Environment Setup

Figure 3.1 depicts .env for docker configurations which denotes resource limits for workers and other configurations for spark master and worker services.

```
# spark config
SPARK_IMAGE=apache/spark:3.5.1

# spark ports
SPARK_MASTER_PORT=7077
SPARK_MASTER_UI_PORT=8080

# Res limits
SPARK_WORKER_CORES=2
SPARK_WORKER_MEMORY=2G

# local dir
DOCKERDIR=./data
SRCDIR=./src
DIDIR=./di
```

Figure 3.1: .env file for docker config

Step 3.2: PySpark Distributed Job Implementation

Here our main usecase is compute network matrices and we start with parse the text file into a Spark Dataframe while stripping out metadata headers as depicted in figure 3.2 below.

```
def load_data(self, file_path: str) -> DataFrame:
    raw_df = self.spark.read.text(file_path)

    # filter start with #
    filtered_df = raw_df.filter(~col("value").startswith("#"))

    # Edges check just to confirm
    total_edges = filtered_df.count()
    self.logger.info(f"Total Edges: {total_edges}")

    # split -> spaces or tabs
    split_col = split(col("value"), r"\s+")

    return (filtered_df.select(
        split_col.getItem(0).cast("int").alias("from_node"),
        split_col.getItem(1).cast("int").alias("to_node")
    ))
```

Figure 3.2: parse to spark data frame

Next we have implementation of in-degree distribution calculation as depicted in figure 3.3 below. Here we group the dataset by 'to_node' vertex and count the incoming links pointing to each which is 617094. Here we explicitly say spark to store the computed dataframe in memory which can be reuse on subsequent operations.

```
def count_incoming_links(self, df: DataFrame) -> DataFrame:
    # group -> to_node for counting
    count_df = df.groupBy("to_node").count()

    # Read from RAM if exist, like below read top 50
    count_df.cache()
    return count_df
```

Figure 3.3: Count incoming links

Finally we find top 50 destination nodes as depicted in the figure 3.4 below.

```
def sort_records(self, df: DataFrame, limit: int = 50) -> DataFrame:
    # get top 50 by sorting
    return df.orderBy("count", ascending=False).limit(limit)
```

Figure 3.4: Get top 50 records

Step 3.3: Execution Metrics Capture

We excute the job inside running docker container on a spark cluster.

```
docker exec -it spark-master /opt/spark/bin/spark-submit --master spark://spark-master:7077 /opt/spark/work-dir/src/main.py
```

Top 5 most linked pages

1. 438238 with 84208 links
2. 401873 with 48205 links
3. 184094 with 44290 links
4. 768 with 44101 links
5. 927 with 77067 links

Total time : 9.49 seconds

Dat Skew across worker group : Minimum Skew

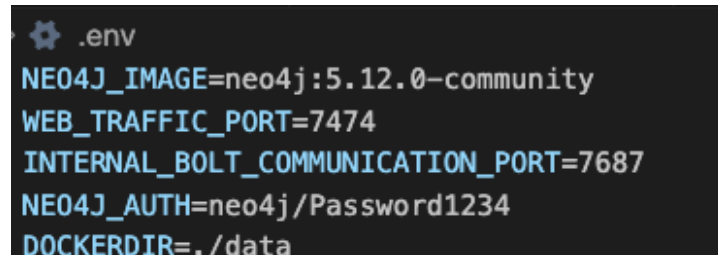
As depicted in figure 3.5 in console log stage tasks completion time mostly falling between 12 ms and 35ms. No tasks took long duration so we can say work was split **uniformly**.

```
2 INFO TaskSetManager: Finished task 160.0 in stage 7.0 (TID 361) in 16 ms on 172.18.0.4
2 INFO TaskSetManager: Starting task 149.0 in stage 7.0 (TID 366) (172.18.0.3, executor 1
s)
2 INFO TaskSetManager: Finished task 147.0 in stage 7.0 (TID 363) in 12 ms on 172.18.0.3
2 INFO TaskSetManager: Starting task 150.0 in stage 7.0 (TID 367) (172.18.0.3, executor 1
s)
2 INFO TaskSetManager: Finished task 148.0 in stage 7.0 (TID 364) in 16 ms on 172.18.0.3
2 INFO TaskSetManager: Starting task 153.0 in stage 7.0 (TID 368) (172.18.0.3, executor 1
s)
2 INFO TaskSetManager: Finished task 149.0 in stage 7.0 (TID 366) in 14 ms on 172.18.0.3
2 INFO TaskSetManager: Starting task 154.0 in stage 7.0 (TID 369) (172.18.0.3, executor 1
s)
2 INFO TaskSetManager: Finished task 150.0 in stage 7.0 (TID 367) in 13 ms on 172.18.0.3
2 INFO TaskSetManager: Starting task 170.0 in stage 7.0 (TID 370) (172.18.0.4, executor 6
s)
2 INFO TaskSetManager: Finished task 165.0 in stage 7.0 (TID 362) in 35 ms on 172.18.0.4
2 INFO TaskSetManager: Starting task 172.0 in stage 7.0 (TID 371) (172.18.0.4, executor 6
s)
```

Task 4: Graph Database Engineering using Neo4j

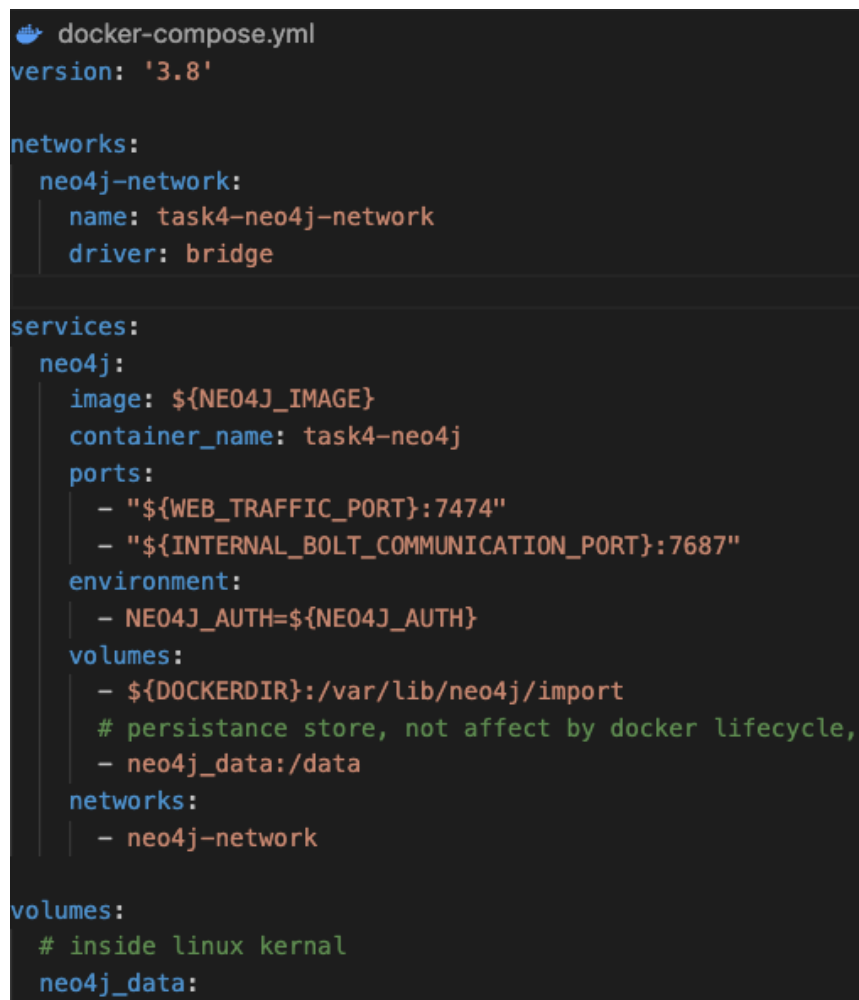
Step 4.1: Neo4j Docker Provisioning

Figure 4.1 below illustrates .env configurations and figure 4.2 below is the docker compose file.

A screenshot of a .env file with a gear icon in the top left corner. The file contains five lines of environment variables in a monospaced font.

```
.env
NEO4J_IMAGE=neo4j:5.12.0-community
WEB_TRAFFIC_PORT=7474
INTERNAL_BOLT_COMMUNICATION_PORT=7687
NEO4J_AUTH=neo4j/Password1234
DOCKERDIR=./data
```

Figure 4.1: .env

A screenshot of a docker-compose.yml file with a Docker icon in the top left corner. The file contains YAML configuration for a Neo4j service, including network settings, service definition with environment variables and ports, and volume definitions.

```
docker-compose.yml
version: '3.8'

networks:
  neo4j-network:
    name: task4-neo4j-network
    driver: bridge

services:
  neo4j:
    image: ${NEO4J_IMAGE}
    container_name: task4-neo4j
    ports:
      - "${WEB_TRAFFIC_PORT}:7474"
      - "${INTERNAL_BOLT_COMMUNICATION_PORT}:7687"
    environment:
      - NEO4J_AUTH=${NEO4J_AUTH}
    volumes:
      - ${DOCKERDIR}:/var/lib/neo4j/import
      # persistence store, not affect by docker lifecycle,
      - neo4j_data:/data
    networks:
      - neo4j-network

volumes:
  # inside linux kernel
  neo4j_data:
```

Figure 4.2: Docker Compose File

Step 4.2: Cypher Graph Ingestion

Data Extraction

Figure 4.3 illustrates the python implementation of subsetting of downloaded patent text structure, number of records(5000) to be extrated would be input by cmd line param.

```
with open(raw_file, "r") as infile, open(out_file, "w") as outfile:

    for line in infile:
        # skip meta
        if line.startswith("#"):
            continue

        parts = line.strip().split()
        if len(parts) == 2:
            from_node, to_node = parts[0], parts[1]
            # write with ',' seperated
            outfile.write(f"{from_node},{to_node}\n")
            count += 1

        if count >= size:
            break
```

Figure 4.3: Data Extraction

Import Logics

Here we used LOAD CSV procedures to convert raw textual records to structured graph node set with directed relationships.

Create Nodes

Figure 4.5 below illustrates the cypher script to create the patent nodes as depicted in figure 4.4

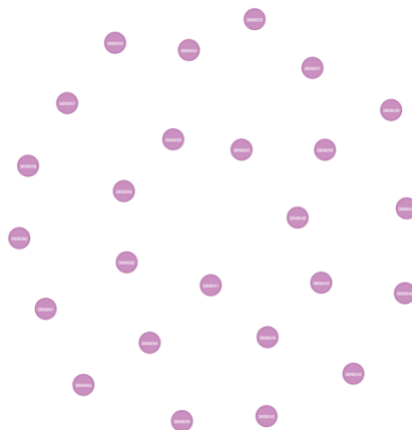


Figure 4.4: Patent Nodes

```

task4.cypher
// 1. Index and make patent id unique
CREATE CONSTRAINT
FOR (p:Patent)
REQUIRE p.id IS UNIQUE;

// 2. Count rows, no need WITH HEADERS we do not write any headers -> output 5000
LOAD CSV FROM 'file:///patent_data_extracted.csv' AS row
RETURN count(row);

// 3. Create All Patent Nodes
LOAD CSV FROM 'file:///patent_data_extracted.csv' AS row
MERGE (:Patent {id: toInteger(row[0])}) // Find or create node from 1st col in row
MERGE (:Patent {id: toInteger(row[1])}) // 2nd col

```

Figure 4.5: Cypher Text for creating the nodes

Create Links

Figure 4.6 illustrates the patent reference relationships which is drawn by the cypher script illustrated in figure 4.7

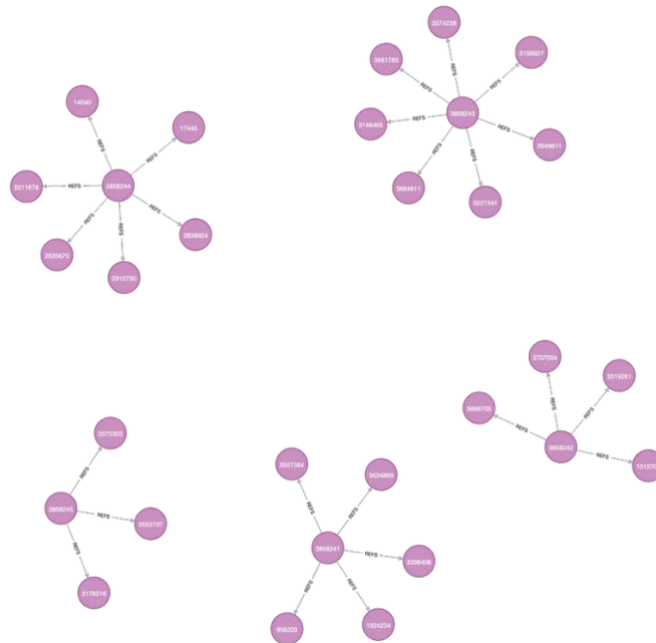


Figure 4.6: Patent Reference drawing

```

// 4. Draw Relationships
LOAD CSV FROM 'file:///patent_data_extracted.csv' AS row
MATCH (fromPatent:Patent {id: toInteger(row[0])}) // Match the existing node
MATCH (toPatent:Patent {id: toInteger(row[1])}) // row 2
MERGE (fromPatent)-[:REFS]->(toPatent);

```

Figure 4.7: Cypher script for creating the patent node links

Step 4.3: Graph Querying and Network Analytics

Identify connected nodes of a target patent

Figure 4.8 depicts the cypher script to identify direct neighbours of patent id 3858243 and figure 4.9 depicts the results.

```
// 5. Find all referenced patents
MATCH (referencingPatent:Patent {id: 3858243})-[:REFS]->(referencedPatent:Patent)
RETURN referencingPatent, referencedPatent;
```

Figure 4.8: Cypher scripts to find all referenced patents



Figure 4.9: Find all referenced parents

Top-tier patent records

Figure 4.10 below depicts the cypher text to find top 5 patent. All top 5 referenced patents has more than 3 citations as shown in figure 4.11 below.

```
// 6. Top 5 most referenced patents (Top 5 has > 2 citations)
MATCH (referencing:Patent)-[:REFS]->(referenced:Patent)
RETURN referenced.id AS Patent_Id, count(referencing) AS Reference_Count
ORDER BY Reference_Count DESC
LIMIT 5;
```

Figure 4.10: Cypher script to find top 5 referenced patents

Patent_Id	Reference_Count
3338819	4
3717571	4
3284344	4
3308054	4
3562800	3

Figure 4.11: Top 5 cited patents

Figure 4.12 depicts the cypher script to find hidden relationships. It searching upto 3 hops (4 patent nodes) and return with maximum hop count length. As the result depicted in the figure 4.13 Patent 2807431 and 3176316 are separated by 4 nodes. The direct search won't return anything which leaves their relationship hidden.

```
// 7. find links with 3 hops
MATCH p = (p1:Patent)-[:REFS*..3]-(p2:Patent)
WHERE
  p1.id < p2.id
  // p1 and p2 have no ref between
  AND
  NOT EXISTS { (p1)-[:REFS]-(p2) }
RETURN p, length(p) AS Hops
ORDER BY Hops DESC
LIMIT 10;
```

Figure 4.12: Find links with 3 hops

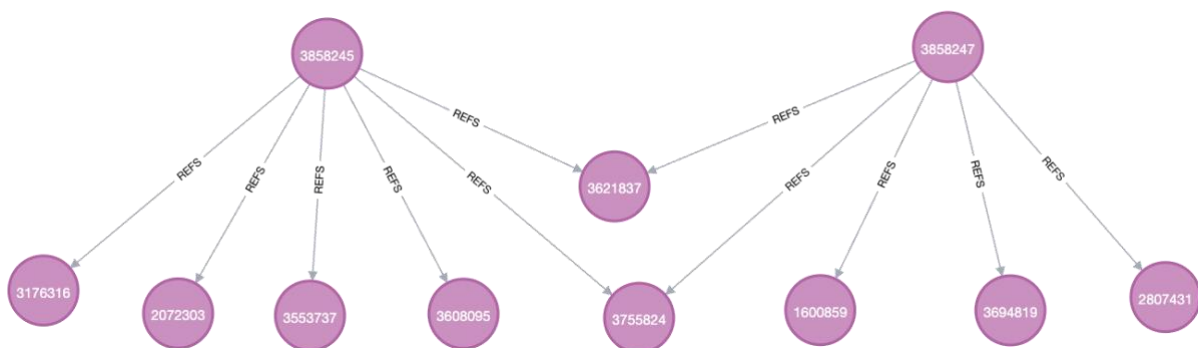


Figure 4.13: Patents links upto 3 hops

Shortest Path display Between nodes 2807431 and 3176316 searching upto depth of 5 hops.

```
// 8. find shortest path between nodes
MATCH (p1:Patent {id: 2807431}), (p2:Patent {id: 3176316})
WHERE NOT EXISTS { (p1)-[:REFS]-(p2) }
MATCH path = SHORTESTPATH ((p1)-[:REFS*..5]-(p2))
RETURN path;
```

Figure 4.14: Find shortest paths between 2 nodes

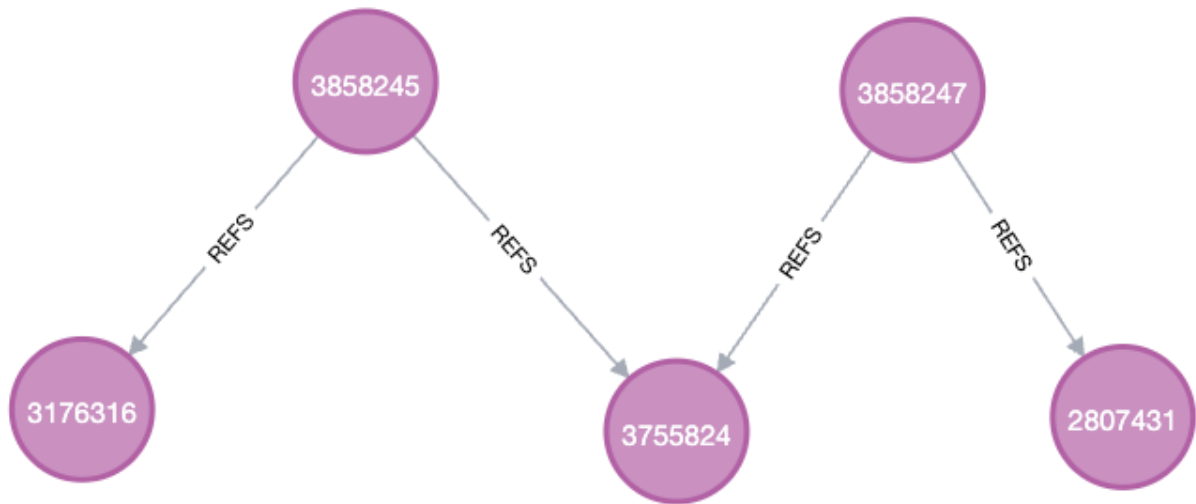


Figure 4.15: Node Graph of shortest path drawn

Task 5: Theoretical Foundations & Future Engineering Trends

Part 5.1: Enterprise Data Governance Frameworks

Introduction

Modern data governance shifting from strict set of rules to more flexible and risk focused strategies backed by technology advancements. Here it more focuss on managing the data accross entire lifecycle rather than individual level. To find, evalualte, address and track potential data threats following four step risk cycle is used.

1. Continous Finding
2. Assessment
3. Management
4. Minitoring

The data protection need to be incoperated from grounds up instead of adding it later.

Centralized vs. Distributed Architectural Patterns

Table 5.1 compares and contrasts the functional limitations of centralized and distributed architectural patterns.

	Centralized(Traditional)	Distributed (Modern)
Movement	Copy data into to a central storage	Zero copying
Frsheness	Batch updates and delayed	Real time access to latest data
Cost	High cost with duplciation	Low cost with no redundant data
Maintainance	Pipeline are fragile	Fewer moving parts and agile
Security	Apply separate across each storage	Apply centrally across all resources

Govern the lifecycle of big data pipelines

GDPR - General Data Protection Regulation

GDPR imposes strict obligations on handling EU citizens data. Compliance requires adherence to principles like lawfulness, purpose limitation, data minimization, accuracy, storage limitation, integrity, and accountability.

The OperationalRole of Metadata Catalog Systems

Metadata systems and data mapping tools are used for identifying where the sensitive data get stored and finding potential risks. Automated Data Lineage & Access Controls [1] allows governance and authorization verified automatically in real time when someone quiries data.

Part 5.2: Emerging Frontiers – Quantum Computing & LLM-Driven Loop Engineering

Quantum Big Data Paradigms

Quantum Machine Learning (QML) and Quantum Computing

Quantum computing operates with principles like superposition and entanglement [2], which allows quantum algorithms to perform complex calculations in parallel, and much faster than classical computers.

Challenge standard constraints on NP-hard processing problems

Quantum computing provides superior processing power which is required for high dimensional calculations and speed up the training of complex models.

Cryptographic signatures

Quantum computing can create unbreakable quantum encryption methods[3] which could make a significant threat to existing classical cryptographic algorithms.

LLM Loop Engineering

Automated Code-Generation Agents in CI/CD & Transformation

Syntax-Constrained Generation: Instead of unconstrained string output the code generation agents use Context Free Grammar (CFG) decoding to enforce strict syntax constraints.

Closed Evaluation Loops : Intermediate outputs from code generation agents are checked using deterministic assertion engines. They would feed errors, stack traces back to agents in case they occur before present to the users.

Open Synthesis

when integrating nondeterministic LLM outputs with strict, deterministic distributed runtime engines we can address following problems.

1. **Non-Determinism[4]:** Different Answer Every Time
When Spark or Flink crashes they can restart and we would get same result which is deterministic.
When we input LLM exact same data twice it may give slightly different answer.
2. **Traffic Jam:** Runs on different speed
Flink or Spark can process millions of records per second, whereas LLM could take significantly higher time just to generate one answer. Therefore sending stream of records to LLM would make a massive bottleneck.
3. The Strict Rule Problem

General frameworks/compiler operates with strict rules[5], even with simple missing paranthesis they would trigger compile/runtime errors. LLMs may output incorrect formats, non exisiting key works which would instantly rejects by compilers. So once they integrated they may trigger erros ins tead of executing the work.

References

1. Wider, A., Verma, S., & Akhtar, A. (2023). "Decentralized Data Governance as Part of a Data Mesh Platform: Concepts and Approaches," *2023 IEEE International Conference on Web Services (ICWS)*.
2. (2025). "Quantum Machine Learning: A Comprehensive Review of Integrating AI with Quantum Computing for Computational Advancements," *PMC*.
3. (2025). "Quantum Machine Learning Models for Enhancing Big Data Analytics," *2025 International Conference on Future Technologies*.
4. Bomma, H. P. (2025). "Integrating Large Language Models into Data Engineering Workflows," *Scholars Journal of Engineering and Technology*.
5. Ding, B., et al. (2024). "Large Language Models as Pretrained Data Engineers: Techniques and Opportunities," *Proceedings of the VLDB Endowment*.